KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
Faculty of Applied Science / Faculty of Information Technology
FINAL EXAMINATION MASTER SOLUTIONS — SEMESTER 1, ACADEMIC YEAR 2026 (SET B)

Course: 060243106 / 060223119 Data Structures and Algorithms | Total Marks: 70 Marks (Normalized to 35% of Grade)
Lecturer: Assoc. Prof. Dr. Pradit Pitaksatheinkul | Status: 100% Comprehensive Step-by-Step Master Solution Key

Question 1: Hashing & Open Addressing with Double Hashing (10 Marks) — Solution

Formula & Parameters: TableSize = 13, R = 11 (largest prime < 13).
$h_0(x) = x \bmod 13$, $f(i) = i \times (11 - (x \bmod 11))$, $h_i(x) = (h_0(x) + f(i)) \bmod 13$.
Sequence of keys inserted: [43, 17, 82, 95, 35, 160].

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Key		160			43	35			95	17	82		
Info		A			A	A			A	A	A		

1.1 Step-by-Step Collision & Probing Trace Table: [4 Marks]

Key (x)	$h_0(x) = x \bmod 13$	Double Hashing Step & Calculation f(i)	Final Index	Collisions
43	43 mod 13 = 4	Empty slot. Placed immediately.	**Index 4**	0
17	17 mod 13 = 4	Collision at 4! f(1)=1×(11-(17 mod 11))=5. h_1=(4+5) mod 13=9 (Empty).	**Index 9**	1
82	82 mod 13 = 4	Collision at 4! f(1)=1×(11-(82 mod 11))=6. h_1=(4+6) mod 13=10 (Empty).	**Index 10**	1
95	95 mod 13 = 4	Collision at 4! f(1)=1×(11-(95 mod 11))=4. h_1=(4+4) mod 13=8 (Empty).	**Index 8**	1
35	35 mod 13 = 9	Collision at 9! f(1)=1×(11-(35 mod 11))=9. h_1=(9+9) mod 13=5 (Empty).	**Index 5**	1
160	160 mod 13 = 4	Collision at 4! h_1=9 (Collision with 17!). f(2)=2×5=10. h_2=(4+10) mod 13=1 (Empty).	**Index 1**	**2**

1.2 Collision Analysis Answer: The key that collided the most is 160 (total 2 collisions, attempted index 4 and index 9 before placing in slot 1).
1.3 Total Collision Count: Total collisions = 0 + 1 + 1 + 1 + 1 + 2 = 6 times.
(Instructor's Warning: Count each probe impact once. Do NOT answer 12! Answer is 6)
1.4 Load Factor Percentage: λ = (6 / 13) × 100% = 46.15%.
1.5 Rehashing Threshold & New TableSize:
- **70% of 13 = 13 × 0.70 = 9.1 ☐ Threshold requires 10 keys. Currently 6 keys are inserted.**
- **Must insert: 10 - 6 = 4 more keys to trigger Rehashing.**
- **New TableSize: First prime > 2 × 13 (26) ☐ TableSize = 29.**

1.6 Concept of Wrap-Around: 'When the calculated hash address reaches or exceeds the last index of the table, it wraps back to index 0 using modulo TableSize arithmetic.'

Question 2: Priority Queue (Binary Min-Heap) Operations — Master Solution (10 Marks)

Given keys: [65, 30, 48, 15, 22, 10, 50, 18, 25, 8].
Array indexing: 1-based (Index 0 is None / Unused Sentinel).

2.1 Array State 1: After sequentially inserting all 10 elements into Min-Heap (Percolate Up): [3 Marks]

Index	0	1	2	3	4	5	6	7	8	9	10	11
Heap Array	None	8	10	15	22	18	48	50	65	25	30	-

2.2 Array State 2: After executing deleteMin() operation #1 (Removes 8, places 30 at root, Percolates Down): [2 Marks]

Index	0	1	2	3	4	5	6	7	8	9	10	11
After Del 1	None	10	18	15	22	30	48	50	65	25	-	-

2.3 Array State 3: After executing insert(12) (Adds 12 at index 10, Percolates Up): [2 Marks]

Index	0	1	2	3	4	5	6	7	8	9	10	11
After Ins 12	None	10	12	15	22	18	48	50	65	25	30	-

2.4 Array State 4: After executing second deleteMin() operation #2 (Removes 10, places 30 at root, Percolates Down): [1 Mark]

Index	0	1	2	3	4	5	6	7	8	9	10	11
After Del 2	None	12	18	15	22	30	48	50	65	25	-	-

2.5 Mathematical Formulas:

- **Left Child Index: 2i | Right Child Index: 2i + 1 | Parent Index: □i / 2□ (Python: i // 2).**

2.6 Truncation Rule (Instructor's Strict Caveat):

Array indices are integers. Integer floor division discards the fractional part (e.g. index 7: 7 // 2 = 3). Rounding up (e.g., 7/2 = 3.5 □ 4) points to the WRONG parent node and violates the Complete Binary Tree property.

2.7 Percolate Down Code Condition:

```
if child != self.current_size and self.heap[child + 1] < self.heap[child]: child += 1
```

Question 3: Insertion Sort Tracing & Position Moves — Master Solution (10 Marks)

3.1 Worst-Case Mathematical Analysis (N = 10 Elements):

- **Worst-Case Configuration: Array sorted in strictly descending order (e.g. [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]).**
- **Formula & Calculation: Total Moves = N(N - 1) / 2 = (10 × 9) / 2 = 45 moves.**
- **Best-Case Configuration: Array already sorted in ascending order □ 0 moves.**

3.2 Pass Tracing on 10-Element Array: [54, 38, 16, 27, 10, 62, 45, 9, 70, 31]

Pass (p)	Key Element	Elements Shifted (Position Moves)	Array State After Pass p
p = 3	27	**Shifts 54 and 38 to right (2 moves)**	**[16, 27, 38, 54, 10, 62, 45, 9, 70, 31]**
p = 6	45	**Shifts 62 and 54 to right (2 moves)**	**[10, 16, 27, 38, 45, 54, 62, 9, 70, 31]**

3.3 Code Modification: Place total_moves += 1 inside the inner while (j >= 0 and arr[j] > key) loop directly adjacent to the element shift statement arr[j + 1] = arr[j].

Question 4: Selection Sort Tracing (Test Program 2 Format) — Master Solution (10 Marks)

Given array: [64, 25, 12, 38, 11, 49, 20].

Round 1:

1. **Find minimum: From unsorted array [64, 25, 12, 38, 11, 49, 20], the minimum is 11 at index 4.**
2. **Swap: swap minimum (11) with first element of unsorted array (64):**
   - **Before swap: [64, 25, 12, 38, 11, 49, 20]**
   - **After swap: [11, 25, 12, 38, 64, 49, 20]**
3. **Result: Now 1 element [11] sorted. The remaining unsorted array is: [25, 12, 38, 64, 49, 20]**

Round 2:

1. **Find minimum: From unsorted array [25, 12, 38, 64, 49, 20], the minimum is 12 at index 2.**
2. **Swap: swap minimum (12) with first element of unsorted array (25):**
   - **Before swap: [11, 25, 12, 38, 64, 49, 20]**
   - **After swap: [11, 12, 25, 38, 64, 49, 20]**
3. **Result: Now 2 elements [11, 12] sorted. The remaining unsorted array is: [25, 38, 64, 49, 20]**

Round 3:

1. **Find minimum: From unsorted array [25, 38, 64, 49, 20], the minimum is 20 at index 6.**
2. **Swap: swap minimum (20) with first element of unsorted array (25):**
   - **After swap: [11, 12, 20, 38, 64, 49, 25]**
3. **Result: Now 3 elements [11, 12, 20] sorted. The remaining unsorted array is: [38, 64, 49, 25]**

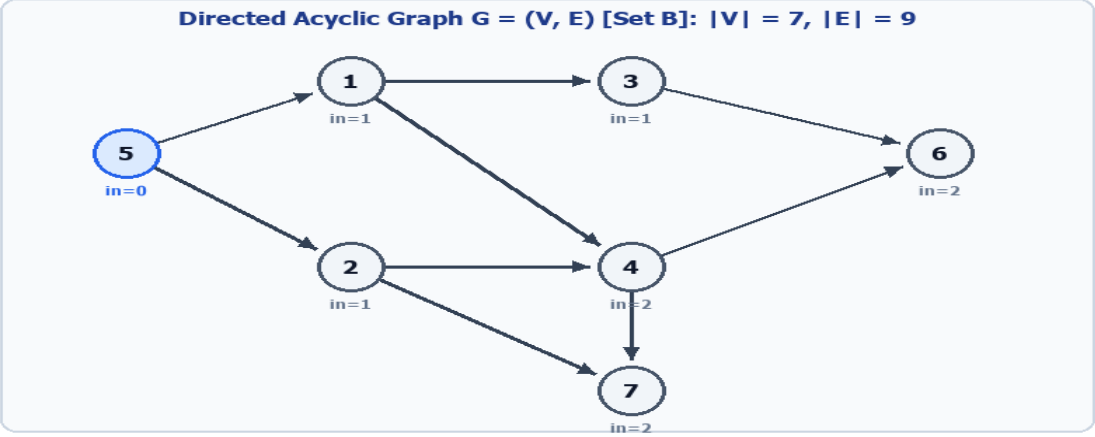
Round 4:

1. **Find minimum: From unsorted array [38, 64, 49, 25], the minimum is 25 at index 6.**
2. **Swap: swap minimum (25) with first element of unsorted array (38):**
   - **After swap: [11, 12, 20, 25, 64, 49, 38]**
3. **Result: Now 4 elements [11, 12, 20, 25] sorted. The remaining unsorted array is: [64, 49, 38]**

4.2 Maximum Swaps: N - 1 = 7 - 1 = 6 swaps. Selection Sort guarantees O(N) memory writes.

Question 5: Topological Sort (Kahn's Algorithm on 7-Vertex DAG) — Master Solution (10 Marks)

DAG: |V| = 7 vertices, |E| = 9 edges. Adjacency: 5:[1,2], 1:[3,4], 2:[4,7], 3:[6], 4:[6,7], 6:[], 7:[].



5.1 In-Degree Array Table: [2 Marks]

Vertex	1	2	3	4	5	6	7
In-Degree	1	1	1	2	0 (Source)	2	2

5.2 Circular Queue Simulation Table: [4 Marks]

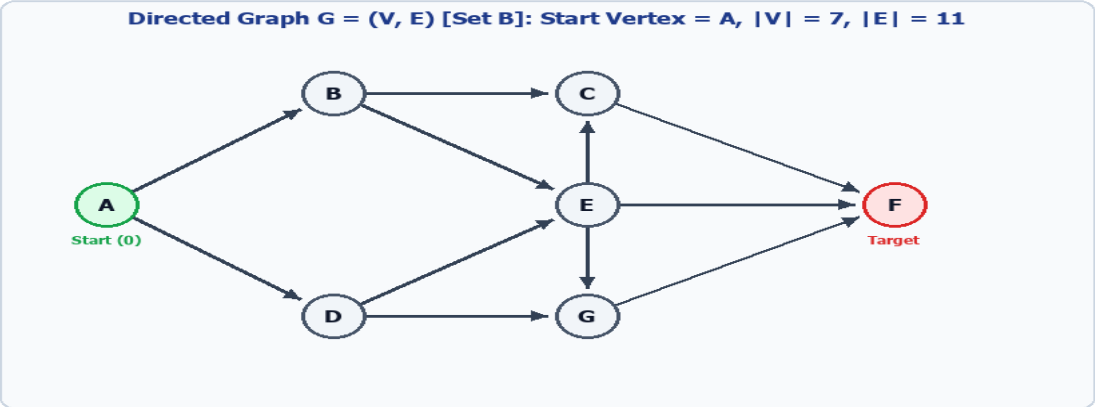
Step #	Action (Dequeue / Enqueue)	Queue Contents	Front Ptr	Back Ptr
Init	Enqueue initial in-degree 0 vertex (5)	[5]	0	0
1	Dequeue 5 (Output 5). Indegree 1☐0 (enq 1), 2☐0 (enq 2)	[1, 2]	1	2
2	Dequeue 1 (Output 1). Indegree 3☐0 (enq 3), 4: 2☐1	[2, 3]	2	3
3	Dequeue 2 (Output 2). Indegree 4☐0 (enq 4), 7: 2☐1	[3, 4]	3	4
4	Dequeue 3 (Output 3). Indegree 6: 2☐1	[4]	4	4
5	Dequeue 4 (Output 4). Indegree 6☐0 (enq 6), 7☐0 (enq 7)	[6, 7]	5	6
6	Dequeue 6 (Output 6)	[7]	6	6
7	Dequeue 7 (Output 7). Queue empty.	[ ] (Empty)	7	6

5.3 Final Topological Ordering: 5, 1, 2, 3, 4, 6, 7 (All 7 vertices processed).

5.4 Directed Cycle Detection: If the graph contains a directed cycle, all vertices within the cycle will continuously maintain in-degree ≥ 1. Consequently, the Queue becomes empty before all |V| vertices are processed. The algorithm detects the cycle because processed_count < |V|.

Question 6: Unweighted Shortest Path Algorithm (BFS) — Master Solution (10 Marks)

Given graph with |V| = 7 vertices (A to G), |E| = 11 edges. Start vertex is Vertex A:



6.1 Shortest Path Processing Table (Assignment 4 Style): [5 Marks]

Vertex (v)	Known (T / F)	Dist (d_v)	Path (p_v)
A (Start)	F ☐ T	999 ☐ 0	0
B	F ☐ T	999 ☐ 1	0 ☐ A
C	F ☐ T	999 ☐ 2	0 ☐ B
D	F ☐ T	999 ☐ 1	0 ☐ A
E	F ☐ T	999 ☐ 2	0 ☐ B
F (Sink)	F ☐ T	999 ☐ 3	0 ☐ C
G	F ☐ T	999 ☐ 2	0 ☐ D

6.2 Circular Queue Tracing: [2 Marks]

Step	Action	Queue State
Init	Enqueue Start Vertex A	[A]
1	Dequeue A (Known=T). Enqueue neighbors B, D (dist=1)	[B, D]
2	Dequeue B (Known=T). Enqueue neighbors C, E (dist=2)	[D, C, E]
3	Dequeue D (Known=T). Enqueue neighbor G (dist=2)	[C, E, G]
4	Dequeue C (Known=T). Enqueue neighbor F (dist=3)	[E, G, F]
5	Dequeue E (Known=T). Neighbors C, F, G already reached	[G, F]
6	Dequeue G (Known=T). Neighbor F already reached	[F]
7	Dequeue F (Known=T). Queue empty	[ ] (Empty)

6.3 Backtracking Analysis Answers:

- **Shortest Path from A to F:**
  Backtrack: F ☐ pv(F)=C ☐ pv(C)=B ☐ pv(B)=A.
  Length: 3 edges | Path: A, B, C, F
- **Shortest Path from A to G:**
  Backtrack: G ☐ pv(G)=D ☐ pv(D)=A.
  Length: 2 edges | Path: A, D, G

(Instructor's Warning: Length is counted in EDGES [integer]. Writing arrows A ☐ B ☐ C ☐ F receives ZERO marks)

Question 7: Advanced Theory, Structural Properties & Percentage Calculations — Master Solution (10 Marks)

7.1 Structural Properties (Two Properties vs Three Properties): [3 Marks]

(a) Data structure with Two Formal Properties: Binary Heap (Priority Queue)

- **Structure Property: Must be a Complete Binary Tree (completely filled at all levels except the lowest level, which is filled from left to right). This allows pointerless 1D array representation.**
- **Heap-Order Property: In a Min-Heap, for every node X, Key(Parent) ≤ Key(X), ensuring the minimum element is ALWAYS located at the Root (array index 1).**

(b) Concept with Three Formal Properties: Ideal Hash Function (Lecture 7)

- **Property 1: Fast Computation: Executes in constant O(1) time.**
- **Property 2: Deterministic: Identical key always hashes to the exact same table slot.**
- **Property 3: Uniform Distribution: Distributes all keys evenly across all table slots without bias.**

(c) Impossible property in practice: Uniform Distribution (Zero Collisions).

- **Reason 1: Pigeonhole Principle: Key space is infinitely or vastly larger than TableSize; collisions are mathematically guaranteed.**
- **Reason 2: Data Distribution Clustering: Real-world data is non-random and naturally clusters around specific patterns.**

7.2 Graph Memory Waste Percentage Calculation: [2 Marks]

Given $|V| = 8$ vertices, $|E| = 14$ edges:

- **Total cells in Adjacency Matrix = $|V|^2 = 8 \times 8 = 64$ cells.**
- **Cells containing 1 (active edges) = 14 cells □ (14 / 64) × 100% = 21.88%.**
- **Cells containing 0 (wasted memory) = 64 - 14 = 50 cells □ (50 / 64) × 100% = 78.13%.**
- **Adjacency List memory = $|V| + |E| = 8 + 14 = 22$ cells.**
- **Percentage of memory saved = ((64 - 22) / 64) × 100% = 65.63% saved.**

Conclusion: For sparse graphs, Adjacency Matrix wastes over 78% of space. Adjacency List is optimal.

7.3 Complete Graph Edge Calculations: [1.5 Marks]

(a) Undirected Complete Graph K_{12}: $E = V(V - 1) / 2 = (12 \times 11) / 2 = 6 \times 11 = 66$ edges. (Exact integer: 66)

(b) Directed Complete Graph without self-loops: $E = V(V - 1) = 12 \times 11 = 132$ directed edges.

(Instructor's Warning: Writing 12*11/2 or formulas receives 0 marks. Answer must be 66)

7.4 Complete Binary Tree Node Bounds: [2 Marks]

For Complete Binary Tree of height h (Root at h = 0): Min = 2^h, Max = $2^{h+1} - 1$:

- **Height h = 12: Minimum nodes = 2^{12} = 4,096 nodes | Maximum nodes = $2^{13} - 1 = 8,192 - 1 = 8,191$ nodes.**
- **Height h = 15: Minimum nodes = 2^{15} = 32,768 nodes | Maximum nodes = $2^{16} - 1 = 65,536 - 1 = 65,535$ nodes.**

(Strict rule: Evaluated exact integers only. Do NOT leave as 2^{12} or 2^{15})

7.5 Instructor Exam Traps & Specific Definitions: [1.5 Marks]

(a) Path sequence notation: In graph theory, a Path is formally defined as an ordered sequence of vertices separated by commas (e.g., A, B, C). Writing arrows (A □ B □ C) represents directed edge relations, violating mathematical convention and receives 0 marks.

(b) Null Graph Query: If asked for the shortest path between two vertices in a Null Graph (V = {}, E = {}), the only correct answer is: 'No path exists because neither vertex exists in the graph.'

***** END OF MASTER SOLUTIONS (SET B) — COMPREHENSIVE STUDY GUIDE *****